

UNIVERSIDAD NACIONAL DE INGENIERÍA
UNIDAD DE POSGRADO - FACULTAD DE INGENIERÍA INDUSTRIAL Y
DE SISTEMAS



PLAN DE DESPLIEGUE MÍNIMO
SPRINT 11

**“APLICACIÓN DE MODELOS DE INTELIGENCIA ARTIFICIAL PARA LA
DETECCIÓN Y PREDICCIÓN PROACTIVA DE FALLAS FÍSICAS EN
INFRAESTRUCTURAS DE RED DE TELECOMUNICACIONES EN ENTORNOS
NOC”**

PARA OBTENER EL GRADO ACADÉMICO DE MAESTRO EN CIENCIAS CON
MENCIÓN EN INTELIGENCIA ARTIFICIAL

ELABORADA POR:
FERNANDO JOEL BLAZ ALEMAN

ASESOR: PhD. Glen Darío Rodríguez Rafael

LIMA, PERÚ
2026

Resumen ejecutivo

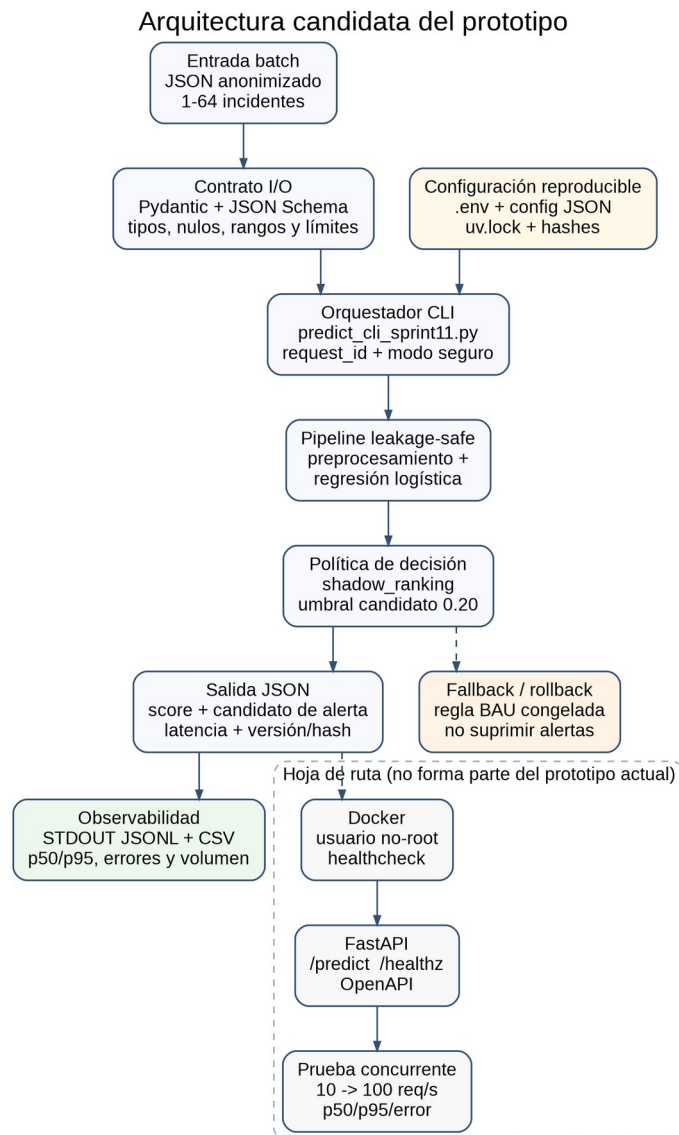
El presente plan convierte el avance experimental de la tesis en un prototipo corrible, trazable y seguro para laboratorio. La arquitectura candidata es un módulo de línea de comandos (CLI) para uso local y batch, debido a que el flujo actual corresponde a analítica interna y no requiere todavía una interfaz web. El artefacto leakage-safe de Sprint 10 se carga una vez por proceso, recibe únicamente información disponible al abrir el incidente, valida tipos y rangos mediante Pydantic y devuelve un score de riesgo, un candidato de alerta, la latencia y la versión/hash del modelo. El modo predeterminado es shadow_ranking: el modelo ordena casos, pero no elimina ni reemplaza las alertas de la regla BAU. La matriz de confusión confirmó que el positivo real es el incumplimiento del OLA y que una alerta es solo una predicción positiva. El umbral original 0.245 redujo falsas alarmas, pero elevó los falsos negativos de 19 a 54; por ello no se propone como reemplazo autónomo. Se documenta un umbral recall-first de 0.20 como candidato posterior al feedback, sujeto a revalidación temporal. El plan incorpora contratos I/O, entorno fijado, lockfile, Makefile, prueba E2E, observabilidad, smoke/golden tests, seguridad, hoja de ruta a Docker/API y rollback inmediato a BAU.

1. Arquitectura candidata

Se selecciona el modo módulo/CLI para el prototipo actual. La API FastAPI y el contenedor Docker quedan en la hoja de ruta, después de validar concurrencia, percepción del stakeholder y contrato de recall.

Figura 1

Arquitectura candidata del prototipo y evolución a Docker/API.



Nota. El modo operativo predeterminado es shadow_ranking; BAU permanece como fallback y fuente de decisión.

2. Contratos de entrada y salida

La clase real positiva corresponde al incidente que efectivamente incumple el OLA. La predicción positiva corresponde a un candidato de alerta. Esta distinción se expresa en el contrato para evitar la confusión observada durante la revisión de la matriz de confusión.

Tabla 1
Campos principales del contrato de entrada.

Grupo	Campos	Validación
Categoricos	domain, area, priority, type_of_incident, trouble_type, incident_type, network_id, reason_group, branch_id	Nulos a MISSING; longitud limitada; categorías nuevas permitidas.
Temporales	year, quarter, month, week_of_year, day_of_week, hour	Rangos válidos y tipos numéricos.
Operativos	is_weekend, is_night, sla_threshold_hours	Binarios 0/1 y OLA entre 0 y 720 horas.
Bloqueados	duration_hours, duration_hours_evidence, resolution_time, end_time, label_source	No forman parte del esquema por fuga o disponibilidad temporal.

```
python src/predict_cli_sprint11.py \  
--input examples/input_valid_sprint11.json \  
--output results/prediction_cli_sprint11.json --pretty
```

La salida incluye request_id, definiciones semánticas, modo de decisión, versión y hash del modelo, latencia de carga, latencia de inferencia, score, candidato de alerta, banda de ranking y umbral. Los errores de entrada se devuelven con code, message, hint y detalle por campo.

3. Reproducibilidad

Tabla 2
Elementos para regenerar el entorno.

Elemento	Implementación
Python	3.13.x
Semilla	42
Dependencias directas	requirements-sprint11.lock.txt
Lockfile transitive	uv.lock
Configuración	config/deployment_sprint11.json
Variables de entorno	.env.example
Comandos	Makefile
Integridad	artifacts/deployment_manifest_sprint11.json con SHA-256

```
make setup
make schemas
make predict
make test
make e2e
```

4. End-to-end en limpio

1. Clonar la rama y entrar a la raíz del repositorio.
2. Crear o activar un entorno Python 3.13.
3. Ejecutar `make setup` o `uv sync --frozen`.
4. Verificar el artefacto `actual_logreg_sprint10.joblib` y su SHA-256.
5. Ejecutar `make test` y luego `make e2e`.
6. Confirmar `success=true` en `results/e2e_summary_sprint11.json`.

Tabla 3
Criterios de éxito del E2E.

Criterio	Resultado esperado
Exit code	0
Contrato de salida	Válido según Pydantic/JSON Schema
Lote	2 predicciones
Scores	Dentro de [0,1]
Integridad	Hash del modelo coincide
Modo	<code>shadow_ranking</code>
Golden	Scores y etiquetas coinciden dentro de tolerancia

5. Observabilidad

Cada invocación registra eventos JSONL y métricas CSV sin almacenar el contenido crudo del incidente. Los campos mínimos son timestamp UTC, request_id, estado, modo, batch, latencia total y media, versión, hash, umbral y código de error. El SLO de 150 ms se aplica a inferencia caliente; el arranque frío de un proceso CLI se reporta por separado.

Tabla 4
Métricas de laboratorio heredadas de Sprint 10.

Modo	p50	p95	Throughput	Errores internos
Modelo precargado, batch 1	7.92 ms	9.92 ms	123.8 req/s	0 observados
Micro-batch 8	7.14 ms/lote	7.76 ms/lote	1,115 registros/s	0 observados
Micro-batch 16	6.85 ms/lote	7.35 ms/lote	2,302 registros/s	0 observados

6. Validación y pruebas

Tabla 5
Pruebas incorporadas.

Prueba	Propósito	Criterio
Smoke válido	Verificar que el sistema respira.	Carga el modelo y devuelve 2 predicciones válidas.
Smoke inválido	Control de errores.	hour=25 genera INPUT_VALIDATION_ERROR.
Borde	Robustez a categorías nuevas.	No produce error interno.
Golden	Estabilidad determinística.	Mismo input, mismos scores dentro de 1e-10.
Fuga	Seguridad metodológica.	El esquema no contiene variables posteriores al resultado.
Matriz	Trazabilidad del feedback.	Reproduce TP/FP/FN/TN de 0.245 y 0.20.

7. Seguridad y configuración

Tabla 6
Variables y límites seguros.

Control	Valor	Justificación
Modo por defecto	shadow_ranking	No altera decisiones reales.
Payload máximo	65 536 bytes	Reduce abuso y errores de memoria.
Batch máximo	64	Coincide con benchmark y contrato.
Timeout/SLO warm	150 ms	Objetivo de laboratorio.
Artefacto	SHA-256 verificado	Detecta corrupción o versión incorrecta.
Datos	Anonimizados/mock	No exponer identificadores operativos.
Secretos	No se versionan	Se parametrizan por entorno.

8. Matriz de confusión y corrección recall-first

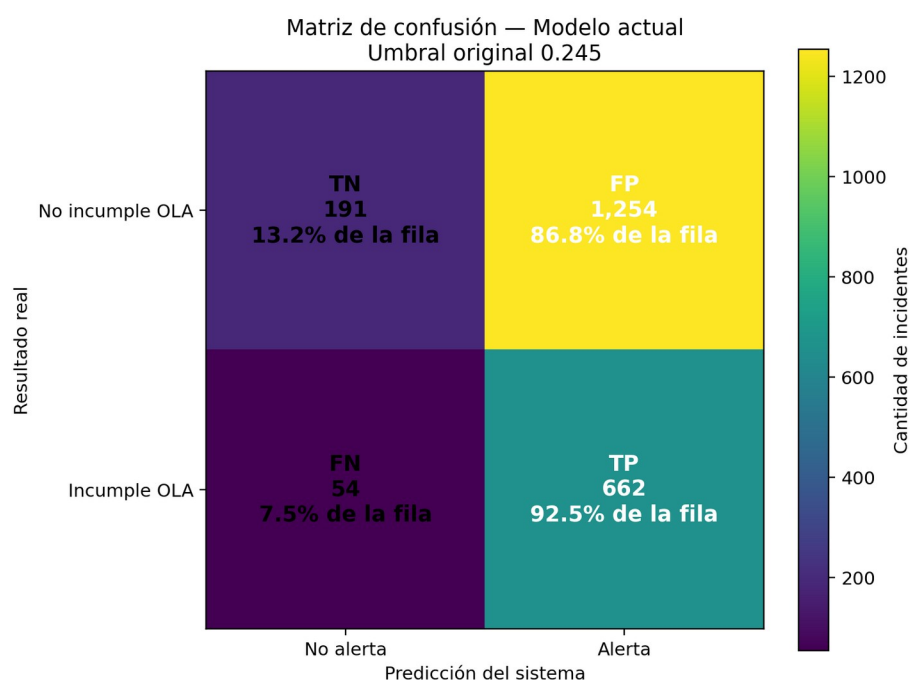
Tabla 7

Comparación de puntos de operación.

Sistema	TN	FP	FN	TP	Recall	Alertas
Baseline BAU 0.50	25	1420	19	697	97.35%	97.96%
Modelo 0.245	191	1254	54	662	92.46%	88.66%
Candidato 0.20	68	1377	18	698	97.49%	96.02%

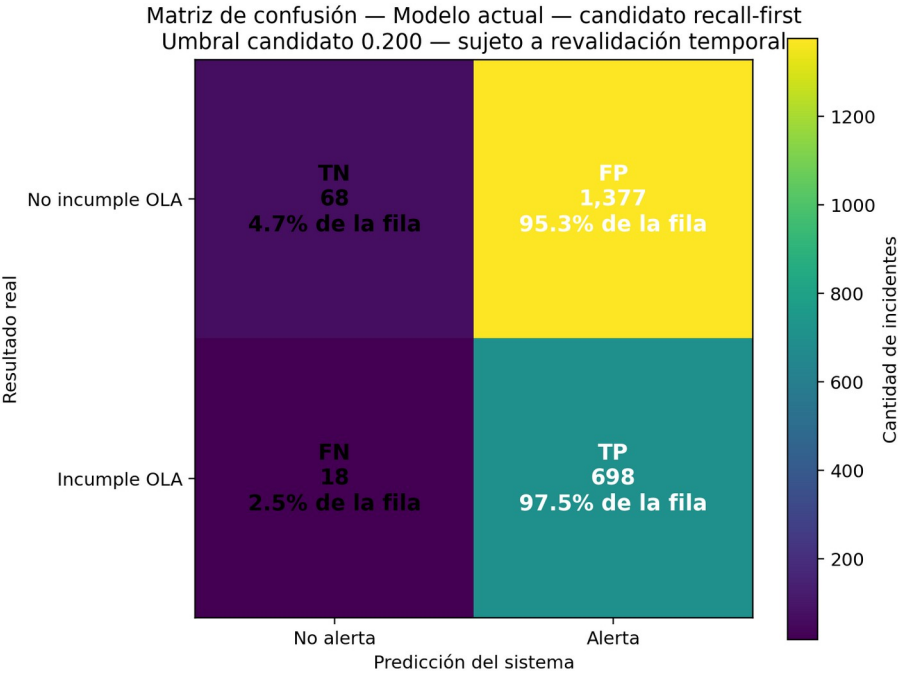
Figura 2

Matriz de confusión del modelo con el umbral original 0.245.



Nota. El aumento de FN explica la caída de recall; este punto no se propone como reemplazo autónomo.

Figura 3
Matriz de confusión del candidato recall-first 0.20.



Nota. El resultado es exploratorio porque el umbral se revisó después del feedback; requiere una ventana temporal nueva.

9. Hoja de ruta a Docker/API

Tabla 8

Tareas, responsables y fechas objetivo.

Tarea	Responsable	Fecha	Criterio
Congelar Sprint 11 y tag	Fernando Blaz	18-07-2026	Tag + manifiesto SHA-256
Dockerfile no-root + healthcheck	Fernando Blaz	21-07-2026	Build reproducible
FastAPI /predict y /healthz	Fernando Blaz	23-07-2026	OpenAPI + tests
Carga 10 → 100 req/s	Fernando Blaz	25-07-2026	p50/p95/error por escalón
Shadow con revisión NOC	Tesista + stakeholder por confirmar	29-07-2026	Bitácora FP/FN
Usabilidad/aceptación	Stakeholder por confirmar	31-07-2026	n≥3; utilidad/claridad ≥4/5
Decisión canary/no-go	Tesista + asesor	03-08-2026	Acta + rollback drill

10. Riesgos, mitigaciones y rollback

Tabla 9

Riesgos principales del prototipo.

Riesgo	Mitigación	Rollback
Caída de recall / FN críticos	Ranking-only, umbral recall-first y revisión de FN.	Deshabilitar modelo y volver a BAU.
Target leakage	Allowlist de features y test automático.	Bloquear release y reentrenar.
Drift temporal o por branch	Monitoreo mensual/slices.	Recalibrar o congelar versión previa.
Dependencias frágiles	Lockfile y hashes.	Restaurar entorno/artefacto etiquetado.
Latencia concurrente	Load test, precarga, batching.	Reducir tráfico o retirar API.
Datos sensibles	Anonimización y logs sin payload.	Detener servicio y sanear evidencia.
Mala UX	A/B guiado y mensajes explícitos.	Mantener BAU y rediseñar salida.

1. Cambiar NOC_MODEL_ENABLED=false o NOC_DECISION_MODE=baseline_only.
2. Detener cualquier acción automática y conservar solo evidencia.
3. Restablecer BAU_rule_frozen_vS7.
4. Ejecutar smoke test del fallback.
5. Registrar versión, hash, causa, ventana y casos afectados.
6. Reactivar únicamente en shadow mode después de repetir tests y carga.